

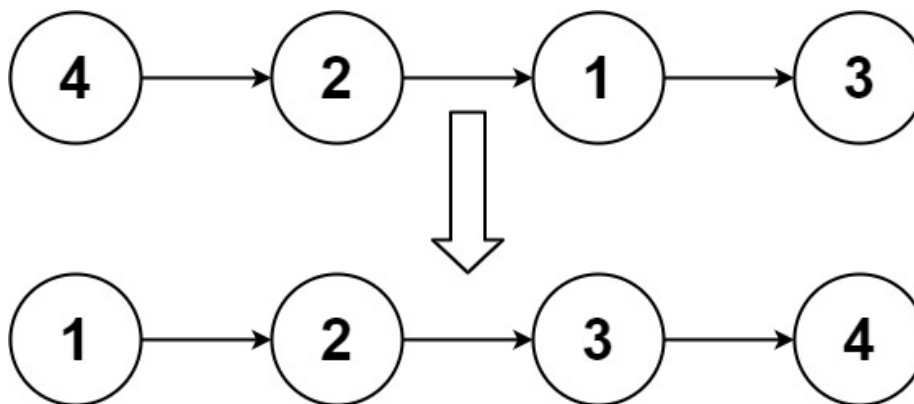
148. 排序链表

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (66.89%)	2504	0	-	-	0

- Tags
- Companies

给你链表的头结点 `head`，请将其按 升序 排列并返回 排序后的链表。

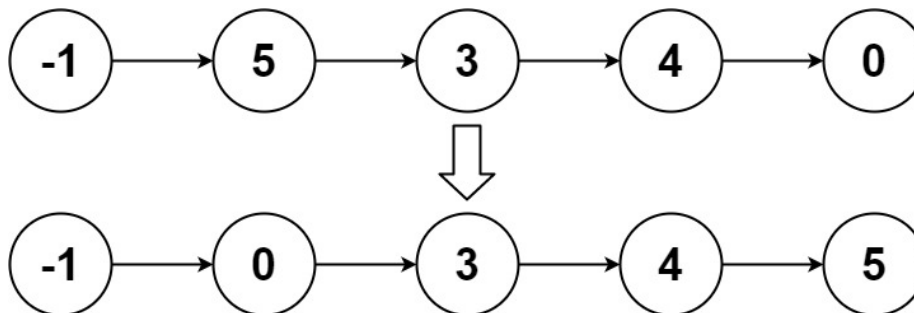
示例 1:



输入: `head = [4,2,1,3]`

输出: `[1,2,3,4]`

示例 2:



输入: `head = [-1,5,3,4,0]`

输出: `[-1,0,3,4,5]`

示例 3:

输入: `head = []`

输出: `[]`

提示:

- 链表中节点的数目在范围 $[0, 5 * 10^4]$ 内
- $105 \leq \text{Node.val} \leq 105$

进阶: 你可以在 $O(n \log n)$ 时间复杂度和常数级空间复杂度下, 对链表进行排序吗?

Discussion | Solution

2. 方法

LeetCode 148: 排序链表 (详细解析)

1. 问题理解

目标: 给定一个单链表的头节点 **head**, 对其进行升序排序。

输入: 单链表的头节点 **head**。

输出: 排序后的链表的头节点。

约束条件 (非常重要):

- 时间复杂度: 要求为 $O(n \log n)$ 。
- 空间复杂度: 要求为 $O(1)$ (常数空间复杂度)。

示例:

- 输入: **head** = [4,2,1,3]
- 输出: [1,2,3,4]
- 输入: **head** = [-1,5,3,4,0]
- 输出: [-1,0,3,4,5]

2. 思路分析: 选择合适的排序算法

- $O(n^2)$ 算法 (如冒泡、插入、选择): 时间复杂度不满足 $O(n \log n)$ 的要求, 直接排除。
- $O(n \log n)$ 算法:
 - 快速排序: 对链表进行快速排序, 原地分区 (partition) 操作比较复杂, 且最坏情况下时间复杂度为 $O(n^2)$, 递归栈空间平均 $O(\log n)$, 最坏 $O(n)$ 。难以稳定地满足 $O(1)$ 空间。
 - 堆排序: 需要将链表转换成类似堆的结构, 或者将值放入数组/堆中再构建链表, 空间复杂度通常不是 $O(1)$ 。
 - 归并排序 (**Merge Sort**): 时间复杂度稳定为 $O(n \log n)$ 。
 - * 自顶向下 (**Top-Down**) 递归法:
 1. 找到链表 midpoint (快慢指针)。
 2. 递归地对左右两半进行排序。
 3. 合并两个已排序的子链表。

- 缺点：递归调用会产生 $O(\log n)$ 的栈空间开销，不满足 $O(1)$ 空间要求。
- * 自底向上 (**Bottom-Up**) 迭代法：
 1. 将链表视为 n 个长度为 1 的有序子链表。
 2. 两两合并相邻的子链表，得到长度为 2 的有序子链表。
 3. 继续两两合并相邻的长度为 2 的子链表，得到长度为 4 的有序子链表。
 4. 重复此过程，每次将子链表长度 `subLength` 翻倍，直到 `subLength` 大于或等于链表总长度 n 。
- 优点：避免了递归，可以通过迭代和精巧的指针操作实现 $O(1)$ 的空间复杂度。

结论：自底向上（迭代）的归并排序是满足时间 $O(n \log n)$ 和空间 $O(1)$ 要求的最佳选择。

3. 自底向上归并排序实现

核心思想

迭代地合并链表中长度为 `subLength` 的有序子段。

步骤

1. 计算链表长度 `length`：遍历一次链表得到总长度。
2. 创建哑节点 `dummy`： `dummy.next = head`，方便处理头节点。
3. 外层循环（控制子链表长度 `subLength`）：
 - 从 `subLength = 1` 开始，每次循环结束后 `subLength *= 2`。
 - 循环直到 `subLength >= length`。
4. 内层循环（遍历链表，合并相邻子链表）：
 - 使用 `prev` 指针记录已排序部分的尾部，`curr` 指针指向当前待处理段的头部。初始 `prev = dummy, curr = dummy.next`。
 - 拆分：从 `curr` 开始，找到长度为 `subLength` 的第一个子链表 `list1`。再从 `list1` 的末尾后面开始，找到长度为 `subLength` 的第二个子链表 `list2`。注意：需要正确断开 `list1` 和 `list2` 以及 `list2` 和后续链表的连接。同时记录下一轮合并的起始点 `nextPairStart`。
 - 合并：调用 `merge(list1, list2)` 函数，将两个有序子链表合并成一个新的有序链表 `merged`。
 - 连接：将 `prev.next` 指向 `merged` 的头节点。
 - 前进 `prev`：将 `prev` 指针移动到 `merged` 链表的末尾。
 - 前进 `curr`：将 `curr` 指针移动到 `nextPairStart`，准备处理下一对子链表。
5. 返回 `dummy.next`。

辅助函数： `merge(list1, list2)`

这个函数与 LeetCode 21 的合并两个有序链表完全相同。

```

private ListNode merge(ListNode list1, ListNode list2) {
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;
    while (list1 != null && list2 != null) {
        if (list1.val <= list2.val) {
            tail.next = list1;
            list1 = list1.next;
        } else {
            tail.next = list2;
            list2 = list2.next;
        }
        tail = tail.next;
    }
    // 连接剩余部分
    tail.next = (list1 != null) ? list1 : list2;
    return dummy.next;
}

```

4. 代码实现

```

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    /**
     * 排序链表 - 自底向上归并排序 ( $O(n \log n)$  时间,  $O(1)$  空间)
     * @param head 链表头节点
     * @return 排序后的链表头节点
     */
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        // 1. 计算链表长度
        int length = 0;
        ListNode node = head;
        while (node != null) {
            length++;
        }
    }
}

```

```

        node = node.next;
    }

    // 2. 创建哑节点
    ListNode dummy = new ListNode(0, head);

    // 3. 外层循环: 子链表长度从 1, 2, 4, ... 递增
    for (int subLength = 1; subLength < length; subLength <= 1) { // subLength *= 2
        ListNode prev = dummy; // 指向每次合并完成后的尾部
        ListNode curr = dummy.next; // 指向当前处理段的头部

        // 4. 内层循环: 遍历链表, 合并相邻的 subLength 长度的子链表
        while (curr != null) {
            // 找到第一个子链表 head1 (长度 subLength)
            ListNode head1 = curr;
            ListNode tail1 = curr; // 用于找到 head1 的尾部
            // 尝试向后移动 subLength - 1 步
            for (int i = 1; i < subLength && tail1 != null && tail1.next != null; i++) {
                tail1 = tail1.next;
            }

            // 如果 tail1 为 null, 说明剩余部分不足一个完整的 head1, 无需合并, 结束内层循环
            if (tail1 == null) {
                break;
            }

            // 找到第二个子链表 head2 (长度 subLength)
            ListNode head2 = tail1.next; // head2 的起始节点
            // 如果 head2 为 null, 说明只剩下 head1, 无需合并, 结束内层循环
            if (head2 == null) {
                break;
            }

            tail1.next = null; // 断开 head1 和 head2

            ListNode tail2 = head2; // 用于找到 head2 的尾部
            // 尝试向后移动 subLength - 1 步
            for (int i = 1; i < subLength && tail2 != null && tail2.next != null; i++) {
                tail2 = tail2.next;
            }

            ListNode nextPairStart = null; // 记录下一对合并的起始点
            if (tail2 != null) {
                nextPairStart = tail2.next;
                tail2.next = null; // 断开 head2 和后续链表
            }
            // 如果 tail2 为 null, 说明 head2 的实际长度可能小于 subLength

```

```

        // 合并 head1 和 head2
        ListNode merged = merge(head1, head2);

        // 将合并后的链表连接到 prev 后面
        prev.next = merged;

        // 将 prev 移动到合并后链表的末尾
        while (prev.next != null) {
            prev = prev.next;
        }

        // 将 curr 移动到下一对的起始点
        curr = nextPairStart;
    }
}

// 5. 返回排序后链表的头节点
return dummy.next;
}

/**
 * 合并两个有序链表 (同 LeetCode 21)
 */
private ListNode merge(ListNode list1, ListNode list2) {
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;
    while (list1 != null && list2 != null) {
        if (list1.val <= list2.val) {
            tail.next = list1;
            list1 = list1.next;
        } else {
            tail.next = list2;
            list2 = list2.next;
        }
        tail = tail.next;
    }
    tail.next = (list1 != null) ? list1 : list2;
    return dummy.next;
}
}

```

5. 代码解析

- 计算长度 **length**: 虽然不是绝对必要 (可以通过判断 **curr** 是否为 **null** 来控制外层循环), 但计算长度有助于理解外层循环的终止条件。

- 哑节点 **dummy**: 使得对头节点的处理与普通节点一致, 简化代码。
- 外层循环 **subLength**: 控制每次合并的子链表的基准长度, 按 1, 2, 4, 8... 增长。
- 内层循环 **curr**: 负责在一次 **subLength** 的迭代中, 遍历整个链表, 找到相邻的 **head1** 和 **head2** 进行合并。
- 寻找 **head1, tail1, head2, tail2**: 通过 **for** 循环和指针移动来确定两个子链表的范围。需要仔细处理边界情况 (如剩余节点不足 **subLength**)。
- 断开连接: 在调用 **merge** 之前, 必须通过 **tail1.next = null** 和 **tail2.next = null** (如果 **tail2** 存在) 将子链表独立出来。同时保存 **nextPairStart**。
- **merge** 函数: 执行实际的合并操作。
- 连接 **prev.next = merged**: 将合并后的有序子链表接回主链表。
- 移动 **prev**: 将 **prev** 移动到刚合并完的链表的末尾, 为下一次连接做准备。
- 移动 **curr = nextPairStart**: 开始处理下一对子链表。

6. 复杂度分析

- 时间复杂度: $O(n \log n)$ 。外层循环执行 $\log n$ 次 (因为 **subLength** 每次翻倍)。内层循环每次需要遍历整个链表 $O(n)$ 来完成所有子链表的合并。所以总时间是 $O(n \log n)$ 。
- 空间复杂度: $O(1)$ 。我们只使用了常数个额外的指针变量 (**dummy, prev, curr, head1, tail1, head2, tail2, nextPairStart** 等) 以及 **merge** 函数内部的常数个指针。没有使用递归栈或与 n 成比例的额外存储空间。

7. 如何在面试中讲解

1. 明确约束条件:
 - “这道题要求在 $O(n \log n)$ 时间和 $O(1)$ 空间内对链表进行排序。”
2. 分析排序算法:
 - “常见的 $O(n^2)$ 算法不满足时间要求。”
 - “ $O(n \log n)$ 算法中, 快速排序对链表实现 $O(1)$ 空间分区困难且不稳定; 堆排序空间不符; 递归归并排序空间为 $O(\log n)$ 。”
 - “因此, 满足所有条件的最佳选择是自底向上 (迭代) 的归并排序。”
3. 解释自底向上归并排序原理:
 - “它的核心思想是迭代地进行合并。初始时, 每个节点看作一个长度为 1 的有序子链表。”
 - “第一轮, 两两合并长度为 1 的子链表, 得到若干长度为 2 的有序子链表。”
 - “第二轮, 两两合并长度为 2 的子链表, 得到长度为 4 的...”
 - “重复这个过程, 每次合并的子链表长度翻倍, 直到整个链表排序完成。”
4. 描述实现细节和指针操作:
 - “我们需要一个外层循环控制合并的子链表长度 **subLength (1, 2, 4...)**”
 - “内层循环遍历链表, 每次找出相邻的两个长度为 **subLength** 的子链表 **list1** 和 **list2**。”
 - “关键在于如何准确地拆分这两个子链表 (通过设置 **tail.next = null**) 并保存下一部分的起始点。”

- “然后调用一个标准的 `merge` 函数（类似 LeetCode 21）来合并 `list1` 和 `list2`。”
 - “合并后，需要将结果连接回主链表，并更新 `prev` 指针到合并后链表的末尾，以及 `curr` 指针到下一对的起始点。”
 - “使用哑节点可以简化对链表头部的处理。”
5. 强调复杂度和正确性：
- “这个方法的时间复杂度是 $O(n \log n)$ 。”
 - “空间复杂度是 $O(1)$ ，因为它避免了递归，只使用了常数个辅助指针。”
 - “需要仔细处理链表末尾不足 `k` 个节点的情况，确保循环能正确终止且不丢失节点。”

找到具有 1 个许可证类型的类似代码